

# Primary and Foreign Key Relationship Cheat Sheet

## Primary and Foreign Key Relationship Cheat Sheet

Bridges Chapter 2 (database architecture) → Chapter 7 (joins) → Chapter 12 (schema design)

---

### What Primary Keys Do

A primary key is the unique identifier for every row in a table. Think of it as the permanent address Chapter 2 describes — no two rows can share it, and it can never be NULL.

- Guarantees each record is distinct (no silent duplicates)
- Gives every row a reliable anchor that other tables can reference
- Typically an integer (id, customer\_id, order\_id) or a natural key (e.g., ISBN)

```
-- The primary key column in a customers table
customer_id INT PRIMARY KEY -- unique, never NULL
```

---

### What Foreign Keys Do

A foreign key is a column in one table that stores the primary key value of a related row in another table. This is the mechanism behind every JOIN you will ever write.

- Creates the relationship that makes cross-table queries possible
- Must match a value that already exists in the referenced table (referential integrity)
- Can be NULL if the relationship is optional (e.g., an order with no assigned sales rep)

```
-- orders references customers through a foreign key
customer_id INT REFERENCES customers(customer_id)
```

---

### The Key Relationship at a Glance

| Concept     | Lives In                       | Role in a JOIN                          |
|-------------|--------------------------------|-----------------------------------------|
| Primary Key | Parent table (e.g., customers) | The value being matched — always unique |
| Foreign Key | Child table (e.g., orders)     | The reference that points back to       |

|                   |            |                                               |
|-------------------|------------|-----------------------------------------------|
|                   |            | the parent                                    |
| JOIN condition    | Your query | ON orders.customer_id = customers.customer_id |
| Missing condition | Your query | Cartesian product — every row × every row     |

---

# How Keys Become JOIN Conditions

Chapter 7 explains that a JOIN temporarily combines rows from two tables using a shared key. The primary-to-foreign-key link is exactly that shared key.

```
-- INNER JOIN: only customers who have placed orders
SELECT customers.name, orders.order_date
FROM customers
INNER JOIN orders ON customers.customer_id = orders.customer_id;

-- LEFT JOIN: all customers, even those with no orders yet
SELECT customers.name, orders.order_date
FROM customers
LEFT JOIN orders ON customers.customer_id = orders.customer_id;
```

Rule: The ON clause almost always connects a primary key on one side to a foreign key on the other. If you cannot identify which column is which, check the schema documentation (Chapter 12).

---

# One-to-Many vs. Many-to-Many

| Relationship Type | Structure                              | Example                            |
|-------------------|----------------------------------------|------------------------------------|
| One-to-One        | One PK ↔ one FK                        | employees ↔ employee_details       |
| One-to-Many       | One PK ↔ many FKs                      | One customer → many orders         |
| Many-to-Many      | Requires a junction table with two FKs | Students ↔ courses via enrollments |

A junction table holds no primary key of its own domain — its purpose is to pair two foreign keys, enabling you to chain joins across three tables as shown in Chapter 7's multi-table join examples.

---

# Keys Inside Star and Snowflake Schemas (Chapter 12)

- Fact tables hold foreign keys pointing to every surrounding dimension table, plus numeric measures (sales amount, quantity).
- Dimension tables hold the primary keys that fact tables reference (customer, product, date).
- In a star schema, you join the fact table directly to each dimension — one hop per join.

- In a snowflake schema, dimensions are themselves normalised into sub-dimensions, adding extra join hops.

```
-- Star schema join pattern
FROM fact_sales
JOIN dim_customer ON fact_sales.customer_id = dim_customer.customer_id
JOIN dim_product  ON fact_sales.product_id  = dim_product.product_id
JOIN dim_date     ON fact_sales.date_id     = dim_date.date_id
```

---

## Quick Diagnostic Checklist

Use this before writing any multi-table query:

- ☐ Identify the primary key of every table you are joining
- ☐ Identify the foreign key that links child to parent
- ☐ Confirm the data types match (joining INT to VARCHAR silently breaks results)
- ☐ Choose the correct join type — INNER loses unmatched rows; LEFT keeps them
- ☐ Verify you have an ON condition for every join to avoid a Cartesian product
- ☐ Cross-reference the schema diagram or data dictionary if the relationship is unclear

---

Primary and foreign keys are not just database housekeeping — they are the logical skeleton that every JOIN traces. Master the key relationships in your schema and the queries follow naturally.